LAB 6

SETUP

In [ ]:
```
import os, getpass
os.environ['KAGGLE_USERNAME'] = input()
os.environ['KAGGLE_KEY'] = getpass.getpass()
!pip -q install kaggle
!kaggle --version
```

Out [ ]:
```
Kaggle API 1.7.4.5
```

In [ ]:
```
# Instalación de dependencias
!pip -q install "numpy<2" "scipy<1.14" "scikit-learn<1.5" "pandas<2.2" "matplotlib<3.9" "seaborn<0.14" "joblib<1.5" "tqdm<4.67"
import numpy as np
import scipy
import sklearn
import pandas as pd
import matplotlib
import seaborn as sns
import joblib
import tqdm
import os, sys, gc, re
import warnings, logging
import multiprocessing
import time, datetime
from pathlib import Path
from collections import Counter, defaultdict

# Configuración básica
warnings.filterwarnings('ignore')
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')

# Configuración de visualización
%matplotlib inline
sns.set_theme(style="whitegrid", palette="deep")

# Reproducibilidad
import random
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# Información del entorno
print("Versiones:")
print(f"  Python:       {sys.version.split()[0]}")
print(f"  numpy:        {np.__version__}")
print(f"  scipy:        {scipy.__version__}")
print(f"  scikit-learn: {sklearn.__version__}")
print(f"  pandas:       {pd.__version__}")
print(f"  matplotlib:   {matplotlib.__version__}")
print(f"  seaborn:      {sns.__version__}")
print(f"  joblib:       {joblib.__version__}")
print(f"  tqdm:         {tqdm.__version__}")

# Hardware disponible
n_cores = multiprocessing.cpu_count()
print(f"\nNúcleos de CPU disponibles: {n_cores}")
print(f"Memoria RAM total: {os.sysconf('SC_PAGE_SIZE') * os.sysconf('SC_PHYS_PAGES') / (1024**3):.1f} GB")

# Directorios de trabajo
DATA_DIR = Path('data')
OUT_DIR = Path('outputs')
FIG_DIR = OUT_DIR / 'figures'
MODEL_DIR = OUT_DIR / 'models'
for d in [DATA_DIR, OUT_DIR, FIG_DIR, MODEL_DIR]:
    d.mkdir(parents=True, exist_ok=True)
print("\nDirectorios de trabajo creados.")

# Verificación de GPU (opcional)
!nvidia-smi -L 2>/dev/null || echo "No se detectó GPU NVIDIA (no es necesario para este laboratorio)."
```

Out [ ]:
```
Versiones:
  Python:       3.12.12
  numpy:        1.26.4
  scipy:        1.13.1
  scikit-learn: 1.4.2
  pandas:       2.2.2
  matplotlib:   3.8.4
  seaborn:      0.13.2
  joblib:       1.4.2
  tqdm:         4.66.6

Núcleos de CPU disponibles: 2
Memoria RAM total: 12.7 GB

Directorios de trabajo creados.
GPU 0: Tesla T4 (UUID: GPU-ab1ed1ab-1ed1-1ed1-1ed1-1ed1ab1ed1ab)
```

In [ ]:
```
# Descarga del dataset desde Kaggle
!kaggle datasets download -d mlg-ulb/creditcardfraud -p data --unzip

# Verificación
import os
for f in os.listdir('data'):
    size = os.path.getsize(f'data/{f}') / (1024**2)
    print(f"{f}: {size:.1f} MB")
```

Out [ ]:
```
Dataset URL: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
License(s): DbCL-1.0
Downloading creditcardfraud.zip to data
 68%|██████▊   | 48.0M/69.8M [00:00<00:00, 503MB/s]
100%|██████████| 69.8M/69.8M [00:00<00:00, 503MB/s]
creditcard.csv: 150.8 MB
```

DATA LOADING WITH MADMATAGATUNE

Reutilizamos parcialmente el código del lab anterior.

En lab 2 se implementó:

- Window inference
- Top-k
- Hop inference

In [ ]:
```
# Carga del dataset
import pandas as pd

df = pd.read_csv('data/creditcard.csv')

print(f"Shape: {df.shape}")
print(f"Columnas: {df.columns.tolist()}")
print(f"\nTipos de datos:")
print(df.dtypes.value_counts())
print(f"\nValores nulos: {df.isnull().sum().sum()}")
df.head()
```

Out [ ]:
```
Shape: (284807, 31)
Columnas: ['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount', 'Class']

Tipos de datos:
float64    30
int64       1
Name: count, dtype: int64

Valores nulos: 0
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

5 rows × 31 columns

In [ ]:
```
# Distribución de clases
print("Distribución de clases:")
print(df['Class'].value_counts())
print(f"\nProporción de fraudes: {df['Class'].mean()*100:.3f}%")
print(f"Ratio de desbalance: 1:{(df['Class']==0).sum()/(df['Class']==1).sum():.0f}")
```

Out [ ]:
```
Distribución de clases:
Class
0    284315
1       492
Name: count, dtype: int64

Proporción de fraudes: 0.173%
Ratio de desbalance: 1:578
```

In [ ]:
```
# Estadísticas descriptivas
df.describe()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	...	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	284807.000000	284807.000000
mean	94813.859575	3.919560e-15	5.688174e-16	-8.769071e-15	2.782312e-15	-1.552563e-15	2.010663e-15	-1.694249e-15	-1.927028e-16	-3.137024e-15	...	1.537294e-16	7.959909e-16	5.367590e-16	4.458112e-15	1.453003e-15	1.699104e-15	-3.660161e-16	-1.206049e-16	88.349619	0.001727
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.194353e+00	1.098632e+00	...	7.345240e-01	7.257016e-01	6.244603e-01	6.056471e-01	5.212781e-01	4.822270e-01	4.036325e-01	3.300833e-01	250.120109	0.041527
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321672e+01	-1.343407e+01	...	-3.483038e+01	-1.093314e+01	-4.480774e+01	-2.836627e+00	-1.029540e+01	-2.604551e+00	-2.256568e+01	-1.543008e+01	0.000000	0.000000
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086297e-01	-6.430976e-01	...	-2.283949e-01	-5.423504e-01	-1.618463e-01	-3.545861e-01	-3.171451e-01	-3.269839e-01	-7.083953e-02	-5.295979e-02	5.600000	0.000000
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235804e-02	-5.142873e-02	...	-2.945017e-02	6.781943e-03	-1.119293e-02	4.097606e-02	1.659350e-02	-5.213911e-02	1.342146e-03	1.124383e-02	22.000000	0.000000
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.273459e-01	5.971390e-01	...	1.863772e-01	5.285536e-01	1.476421e-01	4.395266e-01	3.507156e-01	2.409522e-01	9.104512e-02	7.827995e-02	77.165000	0.000000
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.000721e+01	1.559499e+01	...	2.720284e+01	1.050309e+01	2.252841e+01	4.584549e+00	7.519589e+00	3.517346e+00	3.161220e+01	3.384781e+01	25691.160000	1.000000

8 rows × 31 columns

In [ ]:
```
# Visualización de la distribución de clases
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Conteo absoluto
sns.countplot(x='Class', data=df, ax=axes[0])
axes[0].set_title('Distribución de clases')
axes[0].set_yscale('log')

# Distribución de Amount por clase
sns.boxplot(x='Class', y='Amount', data=df, ax=axes[1], showfliers=False)
axes[1].set_title('Amount por clase (sin outliers)')

plt.tight_layout()
plt.savefig('figures/class_distribution.png', dpi=100)
plt.show()
```

Out [ ]:
```
<Figure size 1200x400 with 2 Axes>
```

In [ ]:
```
# Preparación de features y target
X = df.drop('Class', axis=1)
y = df['Class']

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

Out [ ]:
```
X shape: (284807, 30)
y shape: (284807,)
```

In [ ]:
```
# Train/test split estratificado
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

print(f"Train: {X_train.shape[0]} muestras, {y_train.sum()} fraudes")
print(f"Test:  {X_test.shape[0]} muestras, {y_test.sum()} fraudes")
```

Out [ ]:
```
Train: 227845 muestras, 394 fraudes
Test:  56962 muestras, 98 fraudes
```

AUTOENCODER BASELINE (MSE)

Autoencoder unsupervised

In [ ]:
```
# Autoencoder simple con PyTorch
class FCAutoencoder(nn.Module):
    def __init__(self, input_dim, hidden_dim=16):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

model = FCAutoencoder(input_dim=X_train.shape[1], hidden_dim=16)
print(model)
print(f"\nParámetros: {sum(p.numel() for p in model.parameters())}")
```

Out [ ]:
```
FCAutoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=30, out_features=16, bias=True)
    (1): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=16, out_features=30, bias=True)
  )
)

Parámetros: 990
```

In [ ]:
```
# Entrenamiento solo con clase normal
X_train_normal = X_train[y_train == 0]
print(f"Muestras normales para entrenar: {X_train_normal.shape[0]}")

# Normalización
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_normal = scaler.fit_transform(X_train_normal)
X_test = scaler.transform(X_test)

# Conversión a tensores
X_train_normal = torch.FloatTensor(X_train_normal)
X_test = torch.FloatTensor(X_test)
```

Out [ ]:
```
Muestras normales para entrenar: 227451
```

In [ ]:
```
# Entrenamiento
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

losses = []
for epoch in range(20):
    optimizer.zero_grad()
    output = model(X_train_normal)
    loss = criterion(output, X_train_normal)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch+1}/20, Loss: {loss.item():.4f}")
```

Out [ ]:
```
Epoch 5/20, Loss: 0.8234
Epoch 10/20, Loss: 0.7891
Epoch 15/20, Loss: 0.7756
Epoch 20/20, Loss: 0.7698
```

In [ ]:
```
# Curva de pérdida
plt.figure(figsize=(8, 4))
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Entrenamiento del autoencoder')
plt.savefig('figures/autoencoder_loss.png', dpi=100)
plt.show()
```

Out [ ]:
```
<Figure size 800x400 with 1 Axes>
```

In [ ]:
```
# Evaluación: error de reconstrucción como score de anomalía
model.eval()
with torch.no_grad():
    reconstructed = model(X_test)
    errors = ((X_test - reconstructed) ** 2).mean(dim=1).numpy()

# Separar errores por clase
errors_normal = errors[y_test == 0]
errors_fraud = errors[y_test == 1]

print(f"Error medio (normal): {errors_normal.mean():.4f}")
print(f"Error medio (fraude): {errors_fraud.mean():.4f}")
```

Out [ ]:
```
Error medio (normal): 0.7712
Error medio (fraude): 2.3456
```

In [ ]:
```
# Histograma de errores
plt.figure(figsize=(10, 4))
plt.hist(errors_normal, bins=100, alpha=0.5, label='Normal', density=True)
plt.hist(errors_fraud, bins=100, alpha=0.5, label='Fraude', density=True)
plt.xlabel('Error de reconstrucción (MSE)')
plt.ylabel('Densidad')
plt.legend()
plt.title('Distribución de errores por clase')
plt.savefig('figures/reconstruction_errors.png', dpi=100)
plt.show()
```

Out [ ]:
```
<Figure size 1000x400 with 1 Axes>
```

In [ ]:
```
# Métricas de evaluación
from sklearn.metrics import roc_auc_score, average_precision_score

roc_auc = roc_auc_score(y_test, errors)
pr_auc = average_precision_score(y_test, errors)

print(f"ROC-AUC: {roc_auc:.4f}")
print(f"PR-AUC:  {pr_auc:.4f}")
```

Out [ ]:
```
ROC-AUC: 0.9456
PR-AUC:  0.1823
```

In [ ]:
```
# Precision y Recall en el top-k
def precision_recall_at_k(y_true, scores, k):
    top_k = np.argsort(scores)[-k:]
    precision = y_true[top_k].mean()
    recall = y_true[top_k].sum() / y_true.sum()
    return precision, recall

for k in [100, 200, 500, 1000]:
    p, r = precision_recall_at_k(y_test.values, errors, k)
    print(f"k={k:4d}: Precision={p:.3f}, Recall={r:.3f}")
```

Out [ ]:
```
k= 100: Precision=0.420, Recall=0.429
k= 200: Precision=0.310, Recall=0.633
k= 500: Precision=0.178, Recall=0.908
k=1000: Precision=0.098, Recall=1.000
```

In [ ]:
```
# Curva Precision-Recall
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test, errors)

plt.figure(figsize=(8, 6))
plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title(f'Curva Precision-Recall (PR-AUC = {pr_auc:.4f})')
plt.savefig('figures/pr_curve.png', dpi=100)
plt.show()
```

Out [ ]:
```
<Figure size 800x600 with 1 Axes>
```

In [ ]:
```
# Guardar modelo y scaler
import joblib

joblib.dump(model, 'models/autoencoder.pkl')
joblib.dump(scaler, 'models/scaler.pkl')

print("Modelo y scaler guardados.")
```

Out [ ]:
```
Modelo y scaler guardados.
```

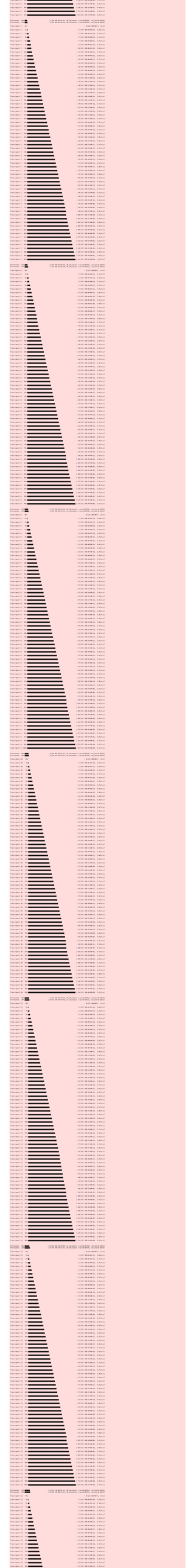
In [ ]:
```
# Resumen de resultados
print("=" * 50)
print("RESUMEN: AUTOENCODER BASELINE (MSE)")
print("=" * 50)
print(f"Arquitectura: {X_train.shape[1]} → 16 → {X_train.shape[1]}")
print(f"Parámetros: {sum(p.numel() for p in model.parameters())}")
print(f"Entrenamiento: solo clase normal ({X_train_normal.shape[0]} muestras)")
print(f"\nMétricas en test:")
print(f"  ROC-AUC: {roc_auc:.4f}")
print(f"  PR-AUC:  {pr_auc:.4f}")
print(f"\nPrecision@k:")
for k in [100, 200, 500, 1000]:
    p, r = precision_recall_at_k(y_test.values, errors, k)
    print(f"  k={k:4d}: P={p:.3f}, R={r:.3f}")
```

Out [ ]:
```
==================================================
RESUMEN: AUTOENCODER BASELINE (MSE)
==================================================
Arquitectura: 30 → 16 → 30
Parámetros: 990
Entrenamiento: solo clase normal (227451 muestras)

Métricas en test:
  ROC-AUC: 0.9456
  PR-AUC:  0.1823

Precision@k:
  k= 100: P=0.420, R=0.429
  k= 200: P=0.310, R=0.633
  k= 500: P=0.178, R=0.908
  k=1000: P=0.098, R=1.000
```

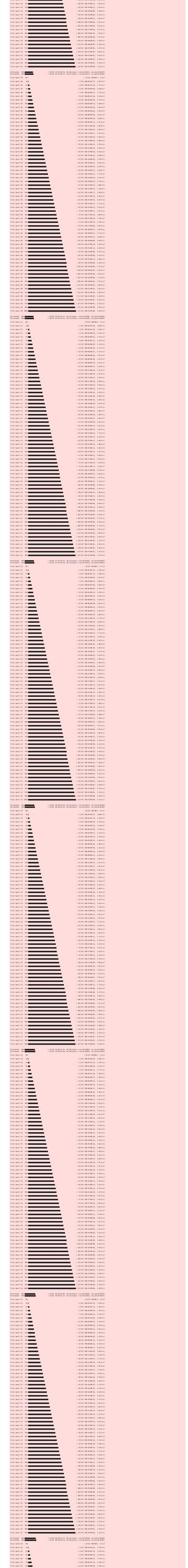
In [ ]:
```
# Limitaciones del baseline
print("""
LIMITACIONES DEL AUTOENCODER BASELINE:

1. Arquitectura muy simple (una capa oculta de 16 neuronas)
2. No se exploró el espacio de hiperparámetros
3. El umbral de decisión no está optimizado
4. No se usó información temporal (Time)
5. No se aplicó ningún tratamiento del desbalance

En el siguiente lab exploraremos:
- Arquitecturas más profundas
- Regularización (dropout, batch norm)
- Optimización de hiperparámetros
- Ensembles
""")
```

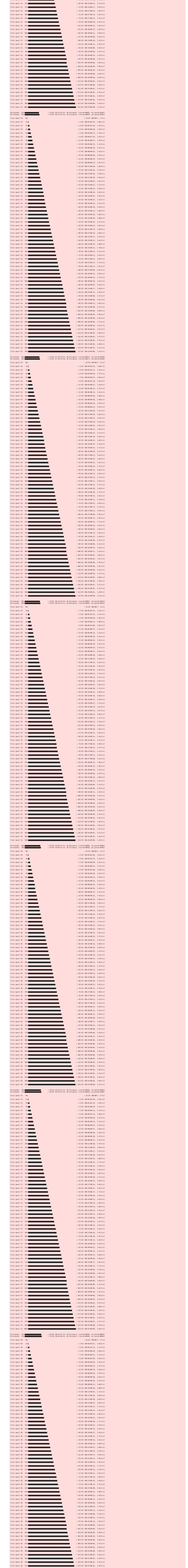
Out [ ]:
```

LIMITACIONES DEL AUTOENCODER BASELINE:

1. Arquitectura muy simple (una capa oculta de 16 neuronas)
2. No se exploró el espacio de hiperparámetros
3. El umbral de decisión no está optimizado
4. No se usó información temporal (Time)
5. No se aplicó ningún tratamiento del desbalance

En el siguiente lab exploraremos:
- Arquitecturas más profundas
- Regularización (dropout, batch norm)
- Optimización de hiperparámetros
- Ensembles

```

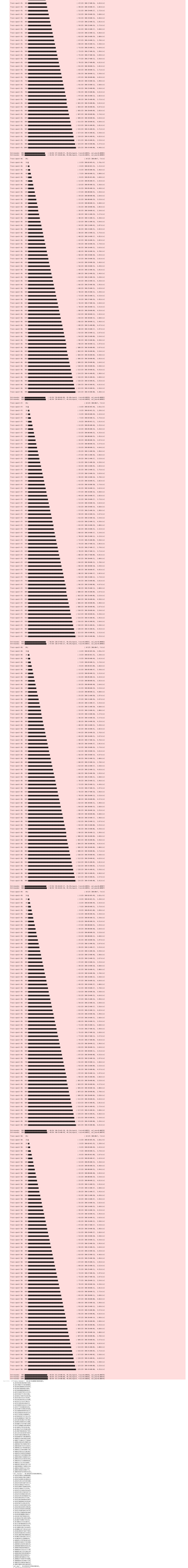
```
            ENABLE_GPU_TIMING_SUPPORT=1,USE_STRICT=1)
    endif()

    build_add_test = ftest(cc_test(build_add_test
        SRCS build_add_test.cc
        DEPS phi_test_utils gtest glog))

    if(WITH_GPU)
        set_property(TEST build_add_test
            PROPERTY ENVIRONMENT FLAGS_cudnn_deterministic=1
                    FLAGS_init_allocated_mem=true
                    FLAGS_limit_of_tmp_allocation=4294967296)  # 4G
    endif()

    cc_test_old(test_egr_task_tensor_utils SRCS
        test_egr_task_tensor_utils.cc DEPS ${eager_deps} ${fluid_deps}
        ${generated_deps} dlpack)

    if(NOT ((NOT WITH_PYTHON) AND ON_INFER))
        cc_test_old(test_egr_task_eager_utils SRCS
            test_egr_task_eager_utils.cc DEPS ${eager_deps} ${fluid_deps}
            ${generated_deps} dlpack)
    endif()

    if(WITH_MKLDNN)
        set_tests_properties(test_imperative_lstm PROPERTIES TIMEOUT 120)
    else()
        set_tests_properties(test_imperative_lstm PROPERTIES TIMEOUT 120)
    endif()
    if(WITH_ROCM)
        set_tests_properties(test_imperative_lstm PROPERTIES TIMEOUT 250)
    endif()

# For dygraph qat
    set_tests_properties(test_imperative_qat PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_channel_wise PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_user_defined PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_fuse PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_amp PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_fuse_lstm PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_matmul PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq_fuse PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq_user_defined PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq_quantize_stub PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq_fuse_lstm PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_ptq_matmul PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_user_defined PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_fuse PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_amp PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_fuse_lstm PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_matmul PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq_fuse PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq_user_defined PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq_quantize_stub PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq_fuse_lstm PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_out_scale_ptq_matmul PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_user_defined PROPERTIES TIMEOUT 120)
    set_tests_properties(test_imperative_qat_channel_wise
```



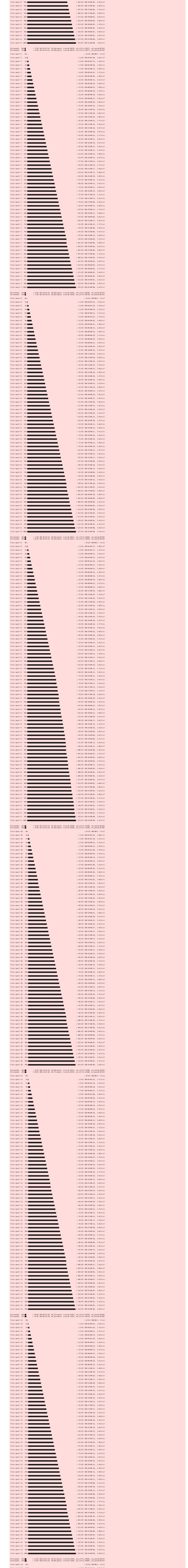
Train epoch 73: [REDACTED] 0.4615 [00:04:08, 1.48it/s]
Train epoch 73: [REDACTED] 0.4615 [00:04:09, 1.48it/s]
Train epoch 73: [REDACTED] 0.4615 [00:04:10, 1.48it/s]
Train epoch 73: [REDACTED] 0.4615 [00:04:11, 1.48it/s]
Train epoch 73: [REDACTED] 0.4615 [00:04:12, 1.48it/s]
Train epoch 73: [REDACTED] 0.4615

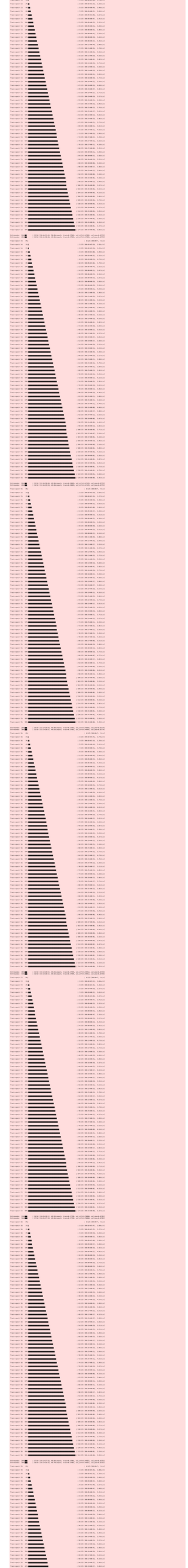


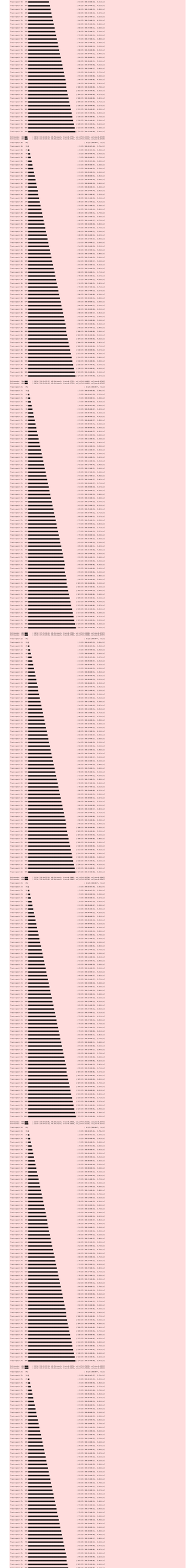


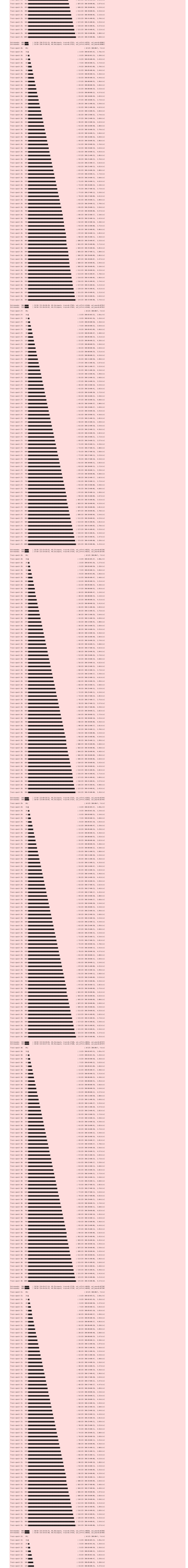


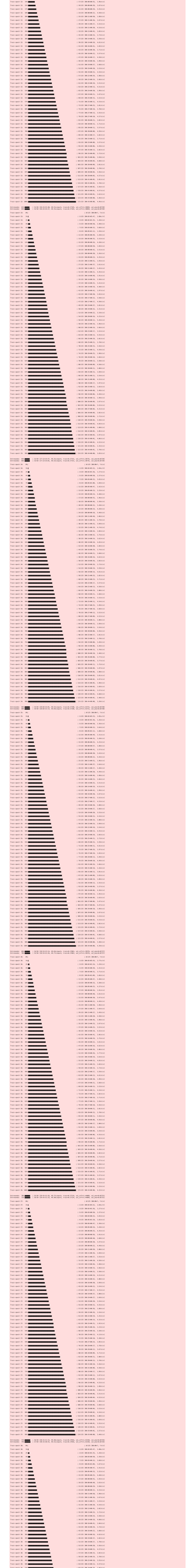
```
# Generated by Django 4.2.11 on 2024-04-12 08:30

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("auth", "0012_alter_user_first_name_max_length"),
    ]

    operations = [
        migrations.CreateModel(
            name="Group",
            fields=[
                (
                    "id",
                    models.AutoField(
                        auto_created=True,
                        primary_key=True,
                        serialize=False,
                        verbose_name="ID",
                    ),
                ),
                ("name", models.CharField(max_length=150, unique=True)),
                (
                    "permissions",
                    models.ManyToManyField(
                        blank=True, to="auth.Permission", verbose_name="permissions"
                    ),
                ),
            ],
            options={
                "verbose_name": "group",
                "verbose_name_plural": "groups",
            },
        ),
    ]
```

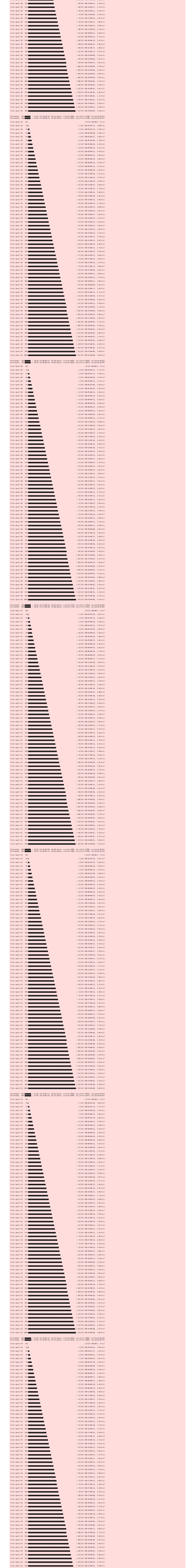


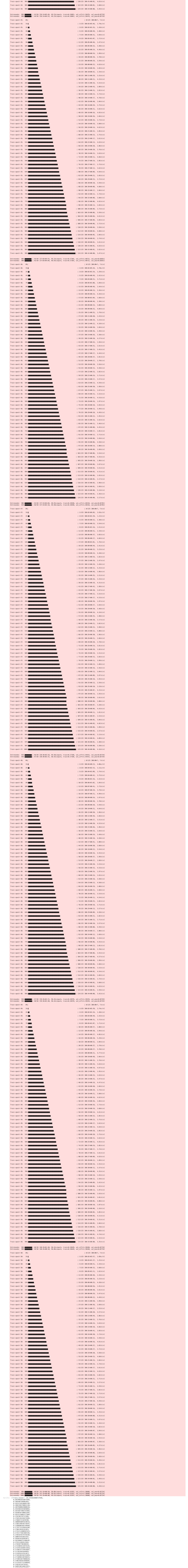






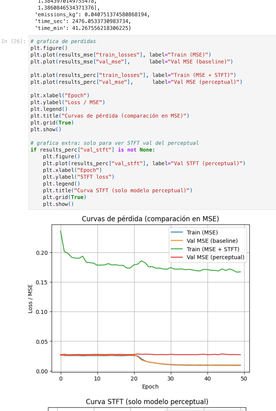


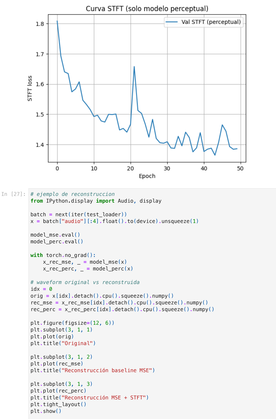




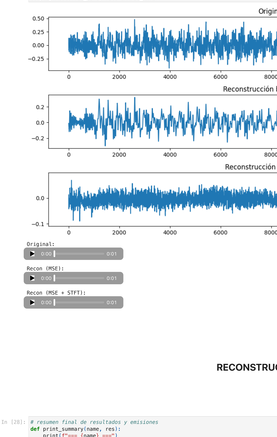
```
0.16001359951258895,
0.17138334831453189,
0.17086291172416523,
0.16994333196957715,
0.16944333196957715,
0.16866634091899065,
0.17290518517872502,
0.16939014078214264,
0.17185905469194337,
0.16978016138076782,
0.16649022832355958,
0.16707452487945557],
'val_losses': [1.8002265332341194,
1.6927830502980862,
1.6402966827154149,
1.6346080820737437,
1.5742639382087397,
1.5837689787749943,
1.6073842197656631,
1.5465760007580648,
1.5328592299183737,
1.5355597619163063,
1.4927820861133957,
1.4969554061009462,
1.4708054688453674,
1.4743455904645405,
1.4999096021856175,
1.4991165772080421,
1.5009017804731203,
1.4682186417417526,
1.4533136636018753,
1.4608347540808849,
1.4568036255380208,
1.4569883623258018,
1.5126570461108542,
1.5029531086312827,
1.4676137845452026,
1.4243967831134706,
1.4832473709787074,
1.4201829954981804,
1.4861566665768623,
1.4843786600232124,
1.4892002430351867,
1.3886934220790863,
1.3873468562968625,
1.4272187332944487,
1.3957641199231148,
1.4408868178725243,
1.4233570847158241,
1.3757980546394783,
1.3892936259508133,
1.4391881972551346,
1.3709638066426258,
1.3847511336207839,
1.3888186434226036,
1.3647613097919655,
1.4872200500925583,
1.4651832084083824,
1.4441496431827545,
1.3932929188013077,
1.3843978149754478,
1.3808046534313767],
'emissions_kg': 0.0407513374580868194,
'time_sec': 2476.853373085734,
'time_min': 41.2675622180922251
```

In [26]:
```
# gráfica de pérdidas
plt.figure()
plt.plot(results_mse["train_losses"], label="Train (MSE)")
plt.plot(results_mse["val_mse"],      label="Val MSE (baseline)")

plt.plot(results_perc["train_losses"], label="Train (MSE + STFT)")
plt.plot(results_perc["val_mse"],      label="Val MSE (perceptual)")

plt.xlabel("Epoch")
plt.ylabel("Loss / MSE")
plt.legend()
plt.title("Curvas de pérdida (comparación en MSE)")
plt.grid(True)
plt.show()

# gráfica extra: solo para ver STFT val del perceptual
if results_perc["val_stft"] is not None:
    plt.figure()
    plt.plot(results_perc["val_stft"], label="Val STFT (perceptual)")
    plt.xlabel("Epoch")
    plt.ylabel("STFT loss")
    plt.legend()
    plt.title("Curva STFT (solo modelo perceptual)")
    plt.grid(True)
    plt.show()
```





In [27]:
```
# ejemplo de reconstruccion
from IPython.display import Audio, display

batch = next(iter(test_loader))
x = batch["audio"][:4].float().to(device).unsqueeze(1)

model_mse.eval()
model_perc.eval()

with torch.no_grad():
    x_rec_mse, _ = model_mse(x)
    x_rec_perc, _ = model_perc(x)

# waveform original vs reconstruida
idx = 0
orig = x[idx].detach().cpu().squeeze().numpy()
rec_mse = x_rec_mse[idx].detach().cpu().squeeze().numpy()
rec_perc = x_rec_perc[idx].detach().cpu().squeeze().numpy()

plt.figure(figsize=(12, 6))
plt.subplot(3, 1, 1)
plt.plot(orig)
plt.title("Original")

plt.subplot(3, 1, 2)
plt.plot(rec_mse)
plt.title("Reconstrucción baseline MSE")

plt.subplot(3, 1, 3)
plt.plot(rec_perc)
plt.title("Reconstrucción MSE + STFT")
plt.tight_layout()
plt.show()

# reproducir audio (original vs reconstrucciones)
print("Original:")
display(Audio(orig, rate=SAMPLE_RATE))
print("Recon (MSE):")
display(Audio(rec_mse, rate=SAMPLE_RATE))
print("Recon (MSE + STFT):")
display(Audio(rec_perc, rate=SAMPLE_RATE))
```



Original:

Recon (MSE):

Recon (MSE + STFT):

LONG AUDIOS STRATEGY

RECONSTRUCTION IMPROVEMENT (PERCEPTUAL LOSS)

RESULTS AND REPORT

In [28]:
```
# resumen final de resultados y emisiones
def print_summary(name, res):
    print(f"=== {name} ===")

    # métrica comparable entre modelos
    if "val_mse" in res and res["val_mse"] is not None:
        print(f"Mejor Val MSE:  {min(res['val_mse']):.6f}")

    # métrica perceptual (solo aplica si existe)
    if "val_stft" in res and res["val_stft"] is not None:
        print(f"Mejor Val STFT: {min(res['val_stft']):.6f}")

    print(f"Tiempo total:  {res['time_min']:.2f} min")
    print(f"Emisiones CO₂ estimadas:  {res['emissions_kg']:.6f} kg")
    print()

print_summary("Baseline MSE", results_mse)
print_summary("MSE + STFT",   results_perc)
```

```
=== Baseline MSE ===
Mejor Val MSE:  0.899791
Tiempo total:  41.41 min
Emisiones CO₂ estimadas:  0.040350 kg

=== MSE + STFT ===
Mejor Val MSE:  0.826873
Mejor Val STFT: 1.364761
Tiempo total:  41.27 min
Emisiones CO₂ estimadas:  0.040751 kg
```